

```

From Coq Require Import Arith Lia Reals ZArith Ring Lia.
From Coq Require Import Arith.PeanoNat.
(* ===== *)
(* Global-normalization reading of Dedenko's manuscript. *)
(* Parameters m,p range over the reals; parity arguments are *)
(* recovered by specialising to integers. A single real factor *)
(* o>1 is postulated to serve all putative counterexamples, and *)
(* the principle of maximum coverage selects the unique choice *)
(* o = 2, restricting the admissible exponents to  $n \in \{1,2\}$ . *)
(* ===== *)

```

```

(* ----- Auxiliary binomial divisibility facts ----- *)

```

```

Local Open Scope Z_scope.

```

```

Lemma Z_sub_split (a b c : Z) : a - b = (a - c) + (c - b).

```

```

Proof. ring. Qed.

```

```

Lemma Zpow_diff_divides (x y : Z) (n : nat) :

```

```

  Z.divide (x - y) (Z.pow x (Z.of_nat n) - Z.pow y (Z.of_nat n)).

```

```

Proof.

```

```

  induction n as [|n IH].

```

```

  - simpl. exists 0%Z; ring.

```

```

  - replace (Z.of_nat (S n)) with (Z.of_nat n + 1) by lia.

```

```

    rewrite !Z.pow_add_r by lia.

```

```

    rewrite !Z.pow_1_r.

```

```

    set (ux := Z.pow x (Z.of_nat n)) in *.

```

```

    set (uy := Z.pow y (Z.of_nat n)) in *.

```

```

    specialize (IH) as [q Hq].

```

```

    rewrite Z.mul_comm with (n := ux) (m := x).

```

```

    rewrite Z.mul_comm with (n := uy) (m := y).

```

```

    rewrite (Z_sub_split (x * ux) (y * uy) (x * uy)).

```

```

    assert (Hx : x * ux - x * uy = x * (ux - uy)) by ring.

```

```

    assert (Hy : x * uy - y * uy = (x - y) * uy) by ring.

```

```

    rewrite Hx, Hy.

```

```

    rewrite Hq.

```

```

    exists (x * q + uy).

```

```

    ring.

```

```

Qed.

```

```

Lemma odd_n_diff_pow_even

```

```

  (a b : Z) (n : nat) :

```

```

  Nat.odd n = true ->

```

```

  Z.divide 2 (Z.pow (a + b) (Z.of_nat n) - Z.pow (a - b) (Z.of_nat n)).

```

```

Proof.

```

```

  intros Hodd.

```

```

  apply Nat.odd_spec in Hodd.

```

```

  destruct Hodd as [k Hk].

```

```

  subst n.

```

```

  replace (Z.of_nat (2 * k + 1)) with (Z.of_nat (S (2 * k))) by lia.

```

```

specialize (Zpow_diff_divides (a + b) (a - b) (S (2 * k))) as Hdiv.
replace ((a + b) - (a - b)) with (2 * b) in Hdiv by ring.
destruct Hdiv as [q Hq].
exists (b * q).
rewrite Hq.
ring.
Qed.

```

Close Scope Z_scope.

```

(* ----- Real-parameter identities (m,p ∈ ℝ) ----- *)
Local Open Scope R_scope.

```

Lemma sum_diff_from_parameters_R

```

  (n : nat) (m p : ℝ) :
  let z := pow m n + pow p n in
  let x := pow m n - pow p n in
  z + x = 2 * pow m n /\
  z - x = 2 * pow p n.

```

Proof.

```

  intros z x; unfold z, x; split; ring.

```

Qed.

Close Scope R_scope.

```

(* ----- Integer-parameter specialisation (m,p ∈ ℤ) ----- *)
Local Open Scope Z_scope.

```

Lemma sum_diff_from_parameters_Z

```

  (n : nat) (m p : ℤ) :
  let z := m ^ Z.of_nat n + p ^ Z.of_nat n in
  let x := m ^ Z.of_nat n - p ^ Z.of_nat n in
  z + x = 2 * m ^ Z.of_nat n /\
  z - x = 2 * p ^ Z.of_nat n.

```

Proof.

```

  intros z x; unfold z, x; split; ring.

```

Qed.

Corollary parity_condition_Z

```

  (n : nat) (m p : ℤ) :
  let z := m ^ Z.of_nat n + p ^ Z.of_nat n in
  let x := m ^ Z.of_nat n - p ^ Z.of_nat n in
  Z.even (z + x) = true /\
  Z.even (z - x) = true.

```

Proof.

```

  intros z x.

```

```

  destruct (sum_diff_from_parameters_Z n m p) as [Hzx Hxz'].

```

split.

- replace $(z + x)$ with $(2 * m \wedge Z.\text{of_nat } n)$ by exact Hxz.
rewrite Z.even_mul; simpl; reflexivity.
- replace $(z - x)$ with $(2 * p \wedge Z.\text{of_nat } n)$ by exact Hxz'.
rewrite Z.even_mul; simpl; reflexivity.

Qed.

Lemma no_parameters_if_parity_violation (n : nat) (z x : Z) :
Z.even (z + x) = false $\vee$ Z.even (z - x) = false ->

~ (exists m p : Z,
z = m $\wedge$ Z.of_nat n + p $\wedge$ Z.of_nat n /\
x = m $\wedge$ Z.of_nat n - p $\wedge$ Z.of_nat n).

Proof.

- intros Hpar [m [p [Hz Hx]]].
- destruct (sum_diff_from_parameters_Z n m p) as [Hsum Hdiff].
- destruct Hpar as [H1 | H2].
- rewrite Hz, Hx, Hsum in H1.
rewrite Z.even_mul in H1; simpl in H1. discriminate.
- rewrite Hz, Hx, Hdiff in H2.
rewrite Z.even_mul in H2; simpl in H2. discriminate.

Qed.

Lemma no_parameters_if_odd (n : nat) (z x : Z) :
Z.odd (z + x) = true $\vee$ Z.odd (z - x) = true ->

~ (exists m p : Z,
z = m $\wedge$ Z.of_nat n + p $\wedge$ Z.of_nat n /\
x = m $\wedge$ Z.of_nat n - p $\wedge$ Z.of_nat n).

Proof.

- intros Hodd [m [p [Hz Hx]]].
- destruct (sum_diff_from_parameters_Z n m p) as [Hsum Hdiff].
- destruct Hodd as [H1 | H2].
- rewrite Hz, Hx, Hsum in H1.
rewrite Z.odd_mul in H1; simpl in H1. discriminate.
- rewrite Hz, Hx, Hdiff in H2.
rewrite Z.odd_mul in H2; simpl in H2. discriminate.

Qed.

Lemma no_parameters_for_example :

~ (exists m p : Z,
2%Z = m $\wedge$ Z.of_nat 3 + p $\wedge$ Z.of_nat 3 /\
1%Z = m $\wedge$ Z.of_nat 3 - p $\wedge$ Z.of_nat 3).

Proof.

- apply (no_parameters_if_parity_violation 3 2 1).
- now left.

Qed.

Close Scope Z_scope.

(* ----- Checks for the manuscript reconstruction steps (2.1)--(2.13) ----- *)

Local Open Scope nat_scope.

Lemma fermat_equation_rearrange_nat (n x y z : nat) :
 $(x^n + y^n = z^n) \% \text{nat} \rightarrow (z^n - x^n = y^n) \% \text{nat}.$

Proof.

intro H.

rewrite <- H.

lia.

Qed.

Lemma parameter_equations_raise_nat (n m p z x : nat) :

$z = (m^n + p^n) \% \text{nat} \rightarrow$

$x = (m^n - p^n) \% \text{nat} \rightarrow$

$(z^n = (m^n + p^n)^n \wedge$

$x^n = (m^n - p^n)^n) \% \text{nat}.$

Proof.

intros Hz Hx.

subst z x.

split; reflexivity.

Qed.

Lemma parameterized_difference_substitution_nat (n m p y : nat) :

$((m^n + p^n)^n - (m^n - p^n)^n) \% \text{nat} = y^n \% \text{nat} \rightarrow$

let z := $(m^n + p^n) \% \text{nat}$ in

let x := $(m^n - p^n) \% \text{nat}$ in

$(z^n - x^n = y^n) \% \text{nat}.$

Proof.

intro H.

exact H.

Qed.

Lemma pythagorean_parameter_example :

let m := 3 in let p := 2 in

let z := $(m^2 + p^2) \% \text{nat}$ in

let x := $(m^2 - p^2) \% \text{nat}$ in

let y := $(2 * m * p) \% \text{nat}$ in

$(z^2 = x^2 + y^2) \% \text{nat}.$

Proof.

simpl.

reflexivity.

Qed.

(*

Clarified reading of (2.10): the odd binomial core is factored over the real domain as $\text{core} = n * l^n$. This is a real scaling assumption/definition for l , not an integer-divisibility assertion about the core.

*)

Local Open Scope R_scope.

Definition real_core_factorization (n : nat) (core l : R) : Prop :=
 (0 < n)%nat /\ core = INR n * pow l n.

Lemma real_core_factorization_unique_value

(n : nat) (core l1 l2 : R) :
 real_core_factorization n core l1 ->
 real_core_factorization n core l2 ->
 INR n * pow l1 n = INR n * pow l2 n.

Proof.

intros [_ H1] [_ H2].
 rewrite <- H1, <- H2.
 reflexivity.

Qed.

Lemma real_core_factorization_substitutes_y_power

(n : nat) (core l y : R) :
 real_core_factorization n core l ->
 pow y n = 2 * core ->
 pow y n = 2 * INR n * pow l n.

Proof.

intros [_ Hcore] Hy.
 rewrite Hy, Hcore.
 ring.

Qed.

Definition boundary_q_equation (n : nat) (q : R) : Prop :=
 pow q n = INR (2 ^ n).

Lemma boundary_q_two (n : nat) : boundary_q_equation n 2.

Proof.

unfold boundary_q_equation.
 rewrite pow_INR.
 reflexivity.

Qed.

Record LocalScaleGlobalNormalization : Type := {

lsgn_n : nat;
 lsgn_j : R;
 lsgn_q : R;
 lsgn_o : R;
 lsgn_j_non_degenerate : 1 < lsgn_j;
 lsgn_boundary_value : boundary_q_equation lsgn_n 2;
 lsgn_global_normalization : lsgn_o = 2
 }.

Lemma local_scale_record_uses_global_boundary

(s : LocalScaleGlobalNormalization) :
 lsgn_o s = 2 /\ boundary_q_equation (lsgn_n s) 2.

Proof.

split.

- exact (lsgn_global_normalization s).

- exact (lsgn_boundary_value s).

Qed.

Close Scope R_scope.

(* ----- Elementary growth facts on naturals ----- *)

Local Open Scope nat_scope.

Lemma pow2_gt_linear_shift (k : nat) :

$2^{k+3} > 2 * (k+3)$.

Proof.

induction k as [| k IH]; simpl.

- lia.

- replace (S k + 3) with (k + 4) by lia.

replace (2 ^ (S k + 3)) with (2 * 2 ^ (k + 3)) by

replace (S k + 3) with (S (k + 3)) by lia;

rewrite Nat.pow_succ_r by lia;

rewrite Nat.mul_comm; reflexivity).

assert (IH' : $2^{k+3} > 2 * (k+3)$) by exact IH.

assert (Htmp : $2 * 2^{k+3} > 2 * (2 * (k+3))$).

{ apply Nat.mul_lt_mono_pos_l; [lia | exact IH']. }

replace (2 * (2 * (k + 3))) with (4 * (k + 3)) in Htmp by lia.

apply Nat.le_lt_trans with (m := 4 * (k + 3)).

+ lia.

+ exact Htmp.

Qed.

Lemma pow2_gt_linear (n : nat) :

$3 \leq n \rightarrow 2^n > 2 * n$.

Proof.

intros Hn.

destruct (Nat.le_exists_sub 3 n Hn) as [k [Hk _]].

rewrite Hk.

replace (3 + k) with (k + 3) by lia.

apply pow2_gt_linear_shift.

Qed.

Lemma pow_eq_linear_cases (n : nat) :

$2^n = 2 * n \rightarrow n = 0 \vee n = 1 \vee n = 2$.

Proof.

destruct n as [| n].

- simpl. intro H. now left.

- destruct n as [| n].

+ simpl. intro H. right; left; lia.

+ destruct n as [| n].

* simpl. intro H. right; right; lia.

* intro H.

```

    assert (3 <= S (S (S n))) by lia.
    specialize (pow2_gt_linear _ H0) as Hgt.
    rewrite H in Hgt. lia.

```

Qed.

Lemma pow_eq_linear_positive (n : nat) :

$2^n = 2 * n \rightarrow n = 1 \vee n = 2$.

Proof.

intro H.

destruct (pow_eq_linear_cases n H) as [H0 | [H1 | H2]].

- subst n. discriminate.

- now left.

- now right.

Qed.

Lemma pow_INR_natpow (k n : nat) :

$\text{pow} (\text{INR } k) n = \text{INR} (k^n)$.

Proof.

rewrite pow_INR.

reflexivity.

Qed.

Lemma covers_two_nat (n : nat) :

$\text{pow } 2 n = \text{INR} (2^n)$.

Proof.

rewrite pow_INR.

reflexivity.

Qed.

Lemma INR_two_mul_nat (n : nat) :

$(2 * \text{INR } n) \% R = \text{INR} (2 * n)$.

Proof.

rewrite mult_INR.

simpl.

reflexivity.

Qed.

Lemma pow_pow_mul : forall (a : R) (n m : nat),

$\text{pow} (\text{pow } a n) m = \text{pow } a (n * m)$.

Proof.

intros a n m.

induction m as [|m IH]; simpl.

- now rewrite Nat.mul_0_r.

- rewrite IH.

rewrite Nat.mul_succ_r.

rewrite pow_add.

simpl.

rewrite Rmult_comm.

reflexivity.

Qed.

```
(* ----- *)
(* Global normalisation and Fermat's Last Theorem *)
(* ----- *)
Local Open Scope R_scope.
```

Definition covers_with (o : R) (n : nat) := pow o n = 2 * INR n.

```
(* ----- Positive real/integer version of the manuscript equations ----- *)
```

```
Record PositiveRealManuscriptData : Type := {
  prm_n : nat;
  prm_n_pos : (0 < prm_n)%nat;
  prm_m : R;
  prm_p : R;
  prm_l : R;
  prm_m_pos : 0 < prm_m;
  prm_p_pos : 0 < prm_p;
  prm_l_pos : 0 < prm_l;
  prm_x : R;
  prm_y : R;
  prm_z : R;
  prm_x_power_eq :
    pow prm_x prm_n = pow (pow prm_m prm_n - pow prm_p prm_n) prm_n;
  prm_z_power_eq :
    pow prm_z prm_n = pow (pow prm_m prm_n + pow prm_p prm_n) prm_n;
  prm_y_power_eq :
    pow prm_y prm_n = 2 * INR prm_n * pow prm_l prm_n
}.
```

Lemma Rpow_pos_of_pos (a : R) (n : nat) :

0 < a -> 0 < pow a n.

Proof.

intro Ha.

induction n as [| n IH]; simpl.

- lra.

- apply Rmult_lt_0_compat; assumption.

Qed.

Lemma positive_l_scaling_forces_cover

(n : nat) (o l y : R) :

(0 < n)%nat ->

0 < l ->

y = o * l ->

pow y n = 2 * INR n * pow l n ->

covers_with o n.

Proof.

intros _ Hl Hy Hpow.

unfold covers_with.
 subst y.
 rewrite Rpow_mult_distr in Hpow.
 apply Rmult_eq_reg_r with (r := pow l n).
 - exact Hpow.
 - pose proof (Rpow_pos_of_pos l n Hl) as Hpos.
 lra.
 Qed.

Lemma positive_real_record_scaling_forces_cover
 (d : PositiveRealManuscriptData) (o : R) :
 prm_y d = o * prm_l d -> covers_with o (prm_n d).
 Proof.
 intro Hy.
 eapply positive_l_scaling_forces_cover.
 - exact (prm_n_pos d).
 - exact (prm_l_pos d).
 - exact Hy.
 - exact (prm_y_power_eq d).
 Qed.

Section PositiveIntegerManuscriptData.

Local Open Scope Z_scope.

Record PositiveIntegerManuscriptData : Type := {
 pim_n : nat;
 pim_n_pos : (0 < pim_n)%nat;
 pim_m : Z;
 pim_p : Z;
 pim_l : Z;
 pim_m_pos : 0 < pim_m;
 pim_p_pos : 0 < pim_p;
 pim_l_pos : 0 < pim_l;
 pim_x : Z;
 pim_y : Z;
 pim_z : Z;
 pim_x_power_eq :
 Z.pow pim_x (Z.of_nat pim_n) =
 Z.pow (Z.pow pim_m (Z.of_nat pim_n) - Z.pow pim_p (Z.of_nat pim_n))
 (Z.of_nat pim_n);
 pim_z_power_eq :
 Z.pow pim_z (Z.of_nat pim_n) =
 Z.pow (Z.pow pim_m (Z.of_nat pim_n) + Z.pow pim_p (Z.of_nat pim_n))
 (Z.of_nat pim_n);
 pim_y_power_eq :
 Z.pow pim_y (Z.of_nat pim_n) =
 2 * Z.of_nat pim_n * Z.pow pim_l (Z.of_nat pim_n)
 }.

```

Lemma integer_manuscript_y_power_even (n : nat) (y l : Z) :
  Z.pow y (Z.of_nat n) = 2 * Z.of_nat n * Z.pow l (Z.of_nat n) ->
  Z.even (Z.pow y (Z.of_nat n)) = true.

```

Proof.

```
intro H.
```

```
rewrite H.
```

```
replace (2 * Z.of_nat n * Z.pow l (Z.of_nat n))
  with (2 * (Z.of_nat n * Z.pow l (Z.of_nat n))) by ring.
```

```
rewrite Z.even_mul.
```

```
simpl.
```

```
reflexivity.
```

Qed.

End PositiveIntegerManuscriptData.

(* ----- Modular remark: congruence is weaker than equality ----- *)

Section ModularRemark.

Local Open Scope Z_scope.

```

Definition modular_congruent (q a b : Z) : Prop :=
  exists t : Z, a = b + q * t.

```

```

Lemma integer_equality_implies_modular_power_congruence
  (n : nat) (x y z q : Z) :
  Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n) =
  Z.pow z (Z.of_nat n) ->
  modular_congruent q
  (Z.pow z (Z.of_nat n))
  (Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n)).

```

Proof.

```
intro H.
```

```
exists 0%Z.
```

```
rewrite <- H.
```

```
ring.
```

Qed.

Lemma modular_congruence_has_integer_witness

```
(q lhs rhs : Z) :
```

```
modular_congruent q lhs rhs -> exists t : Z, lhs = rhs + q * t.
```

Proof.

```
exact (fun H => H).
```

Qed.

Lemma integer_equality_is_zero_modular_witness

```
(n : nat) (x y z q : Z) :
```

```

Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n) =
  Z.pow z (Z.of_nat n) ->

```

```

exists t : Z,
  t = 0%Z /\
  Z.pow z (Z.of_nat n) =
    Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n) + q * t.

```

Proof.

```

intro H.
exists 0%Z.
split; [reflexivity|].
rewrite <- H.
ring.
Qed.

```

Lemma modular_congruence_not_integer_equality :

```

modular_congruent 5 (Z.pow 3 3) (Z.pow 1 3 + Z.pow 1 3) /\
Z.pow 1 3 + Z.pow 1 3 <> Z.pow 3 3.

```

Proof.

```

split.
- exists 5%Z.
  reflexivity.
- discriminate.

```

Qed.

Record ModularParameterResidues : Type := {

```

  mpr_q : Z;
  mpr_n : nat;
  mpr_m : Z;
  mpr_p : Z;
  mpr_A : Z;
  mpr_B : Z;
  mpr_z : Z;
  mpr_x : Z;
  mpr_mn_residue :
    modular_congruent mpr_q (Z.pow mpr_m (Z.of_nat mpr_n)) mpr_A;
  mpr_pn_residue :
    modular_congruent mpr_q (Z.pow mpr_p (Z.of_nat mpr_n)) mpr_B;
  mpr_z_residue : modular_congruent mpr_q mpr_z (mpr_A + mpr_B);
  mpr_x_residue : modular_congruent mpr_q mpr_x (mpr_A - mpr_B)
}.

```

End ModularRemark.

Lemma covers_with_one_forces_two (o : R) :

```

covers_with o 1%nat -> o = 2.

```

Proof.

```

unfold covers_with.
simpl.
intro H.
lra.
Qed.

```

#[local] Lemma INR_inj_nat (a b : nat) : INR a = INR b -> a = b.

Proof.

intro H; first [apply INR_eq in H | apply eq_INR in H | apply INR_inj in H];
exact H.

Qed.

Lemma two_real_normalizations_imply_nat_power_eq

(o : R) (n m : nat) :

covers_with o n ->

covers_with o m ->

Nat.pow (2 * n) m = Nat.pow (2 * m) n.

Proof.

intros Hn Hm.

unfold covers_with in *.

rewrite INR_two_mul_nat in Hn.

rewrite INR_two_mul_nat in Hm.

assert (Hleft : pow (INR (2 * n)) m = pow o (n * m)).

{ rewrite <- Hn. apply pow_pow_mul. }

assert (Hright : pow (INR (2 * m)) n = pow o (n * m)).

{ rewrite <- Hm. rewrite Nat.mul_comm. apply pow_pow_mul. }

rewrite <- Hleft in Hright.

apply INR_inj_nat.

pose proof (pow_INR_natpow (2 * n) m) as Hpow_n.

pose proof (pow_INR_natpow (2 * m) n) as Hpow_m.

rewrite <- Hpow_n.

rewrite <- Hpow_m.

symmetry.

exact Hright.

Qed.

Lemma covers_with_two_characterisation (n : nat) :

covers_with 2 n -> n = 1%nat $\vee$ n = 2%nat.

Proof.

unfold covers_with.

intro H.

rewrite covers_two_nat in H.

rewrite INR_two_mul_nat in H.

apply INR_inj_nat in H.

apply pow_eq_linear_positive in H.

assumption.

Qed.

Lemma maximum_coverage_as_theorem

(o : R) :

covers_with o 1%nat ->

(forall n, covers_with o n -> n = 1%nat $\vee$ n = 2%nat) $\wedge$ o = 2.

Proof.

intro H1.

```

pose proof (covers_with_one_forces_two o H1) as Ho.
split; [| exact Ho].
intros n Hn.
subst o.
apply covers_with_two_characterisation.
exact Hn.
Qed.

```

Section Global_Normalization.

Variable o : R.

Hypothesis normalization_equation :

```

forall (n x y z : nat),
  (2 < n)%nat ->
  (Nat.pow x n + Nat.pow y n)%nat = Nat.pow z n ->
  covers_with o n.

```

Hypothesis coverage_one : covers_with o 1%nat.

Lemma normalization_parameter_is_two : o = 2.

Proof.

```

destruct (maximum_coverage_as_theorem o coverage_one) as [_ Ho].
exact Ho.

```

Qed.

Lemma normalization_forces_small_exponent :

```

forall n x y z,
  (2 < n)%nat ->
  (Nat.pow x n + Nat.pow y n)%nat = Nat.pow z n -> False.

```

Proof.

```

intros n x y z Hn Heq.
specialize (normalization_equation n x y z Hn Heq) as Hcover.
destruct (maximum_coverage_as_theorem o coverage_one)
  as [Hcov Ho].
specialize (Hcov n Hcover) as [Hn1 | Hn2]; lia.

```

Qed.

End Global_Normalization.

Lemma covers_two_one : covers_with 2 1%nat.

Proof.

```

unfold covers_with; simpl.
lra.

```

Qed.

Lemma covers_two_two : covers_with 2 2%nat.

Proof.

```

unfold covers_with; simpl.
lra.

```

Qed.

Lemma covers_two_only_small (n : nat) :
 covers_with 2 n -> n = 1%nat $\vee$ n = 2%nat.
 Proof.
 apply covers_with_two_characterisation.
 Qed.

Corollary fermat_last_theorem_from_global_normalization :
 (forall (n x y z : nat),
 (2 < n)%nat ->
 (Nat.pow x n + Nat.pow y n)%nat = Nat.pow z n ->
 covers_with 2 n) ->
 forall (n x y z : nat),
 (2 < n)%nat ->
 (Nat.pow x n + Nat.pow y n)%nat = Nat.pow z n -> False.
 Proof.

intros Hnorm n x y z Hn Heq.
 specialize (Hnorm n x y z Hn Heq) as Hcover.
 assert (covers_with 2 1%nat) as Hone by apply covers_two_one.
 specialize (maximum_coverage_as_theorem 2 Hone)
 as [Hrest _].
 specialize (Hrest n Hcover) as [Hn1 | Hn2]; lia.
 Qed.

Corollary fermat_last_theorem_via_maximum_coverage :
 (forall (n x y z : nat),
 (2 < n)%nat ->
 (Nat.pow x n + Nat.pow y n)%nat = Nat.pow z n ->
 pow 2 n = 2 * INR n) ->
 forall (n x y z : nat),
 (2 < n)%nat ->
 (Nat.pow x n + Nat.pow y n)%nat = Nat.pow z n -> False.
 Proof.

intros Hnorm n x y z Hn Heq.
 apply (fermat_last_theorem_from_global_normalization Hnorm n x y z Hn Heq).
 Qed.

(* ----- Minimal p-adic bracket for the universal parameter ----- *)
 Section PadicBracket.

Local Open Scope Z_scope.

Record OPadic := { vp_o : nat -> Z }.

Definition NatOddGreater1 (p : nat) : Prop := (1 < p)%nat.

Definition padic_equation (o : OPadic) (n : nat) : Prop :=

forall p, NatOddGreater1 p -> Nat.odd p = true -> Z.of_nat n * vp_o o p = 0%Z.

Lemma odd_primes_vanish_in_o

(o : OPadic) :

padic_equation o 1%nat ->

padic_equation o 2%nat ->

forall p, NatOddGreater1 p -> Nat.odd p = true -> vp_o o p = 0%Z.

Proof.

intros H1 H2 p Hp Hodd.

specialize (H1 p Hp Hodd).

change (Z.of_nat 1) with 1%Z in H1.

rewrite Z.mul_1_l in H1.

exact H1.

Qed.

Lemma two_adic_normalisation (o : OPadic) :

Z.of_nat 1 * vp_o o 2%nat = 1%Z -> vp_o o 2%nat = 1%Z.

Proof.

intro H.

change (Z.of_nat 1) with 1%Z in H.

now rewrite Z.mul_1_l in H.

Qed.

End PadicBracket.

(* ----- Sanity goals for quick regression checks ----- *)

Goal covers_with 2 3%nat -> False.

Proof.

intro H.

destruct (covers_two_only_small 3%nat H) as [H1|H2]; lia.

Qed.

Goal forall n : nat, (2 < n)%nat -> covers_with 2 n -> False.

Proof.

intros n Hn Hcover.

apply covers_two_only_small in Hcover as [H1|H2]; lia.

Qed.

(* ----- Binary scaling reformulation: $n = (1/2) * 2^n$ ----- *)

Lemma two_pow_eq_two_mul_iff_shift (n : nat) :

$(2^n = 2 * n)\%nat \leftrightarrow (n = 2^{n-1})\%nat$.

Proof.

destruct n as [|n].

- simpl. split; intro H; lia.

- split.

+ intro H.

rewrite Nat.pow_succ_r' in H by lia.

```

    apply Nat.mul_cancel_l in H; [[lia].
    replace (S n - 1)%nat with n by lia.
    symmetry.
    exact H.
+ intro H.
  rewrite Nat.pow_succ_r' by lia.
  replace (S n - 1)%nat with n in H by lia.
  rewrite H.
  reflexivity.
Qed.

```

```

Lemma binary_scaling_roots_only_one_two (n : nat) :
  (n = 2 ^ (n - 1))%nat -> n = 1%nat ∨ n = 2%nat.
Proof.
  intro Hshift.
  apply two_pow_eq_two_mul_iff_shift in Hshift.
  now apply pow_eq_linear_positive in Hshift.
Qed.

```

```

Goal forall n : nat, (n = 2 ^ (n - 1))%nat -> n = 1%nat ∨ n = 2%nat.
Proof.
  exact binary_scaling_roots_only_one_two.
Qed.

```

Time - 4 seconds:

FLT on □ main [!?] took 9.1s ...

→ coqc GlobalNormalization.v

FLT on □ main [!?] took 4.0s ...

→